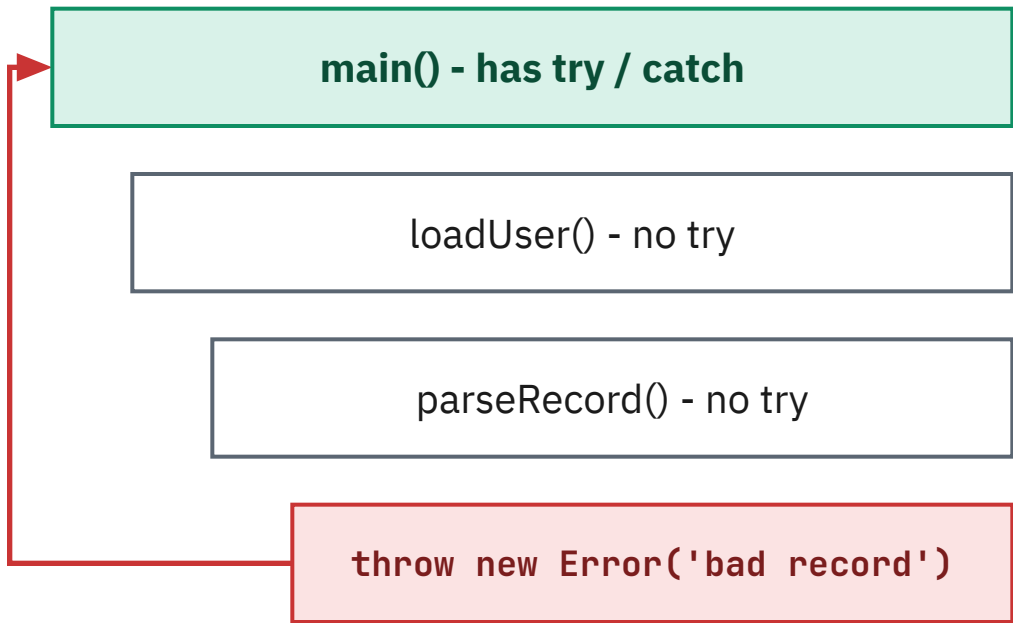


Error Propagation

A throw unwinds the stack until it meets a catch. Frames in between just disappear.



caught here

The two middle frames are abandoned. Nothing after the throw in them ever runs.

Async errors need an async catch

```
try { setTimeout(boom, 0) }  
catch { } // never fires
```

The callback runs later, on an empty stack: the try block is long gone.

```
try { await work() }  
catch (err) { } // works
```

await resumes inside the try, so a rejected promise throws right there.

finally always runs

On success, on throw, even on an early return.
Use it to close, unlock, or hide a spinner,
not to swallow the error.

The Error object

```
err.name  
err.message  
err.stack  
err.cause
```

Rethrow with context

```
throw new Error('Could not load',  
  { cause: err });
```

Always throw an Error, never a string; only an Error carries a stack. Catch where you can actually DO something; letting it travel up is usually right. And a last line of defense: window.onerror plus window.onunhandledrejection.